TP Programmation en langage C

TP Programmation en langage C
Argument d'un programme & Manipulation de fichier texte

I) Les arguments du programme principal

Cette section ne concerne que les environnements, comme UNIX ou MS-DOS, dans lesquels les programmes sont activés en composant une commande de la forme :

```
Mon_prog argument1 argument2 ... argumentk
```

L'exécution d'un programme commence par la fonction main. Tout se passe comme si le système d'exploitation avait appelé cette fonction comme une fonction ordinaire. Il faut savoir que lors de cet appel, des arguments sont fournis au programme. Voici l'en-tête complet de main, en syntaxe ANSI :

```
int main(int argc, char *argv[])
```

avec :

- argc : nombre d'arguments du programme
- argv : tableau de chaines de caractères, qui sont les arguments du programme. Par convention, le premier argument est le nom du programme lui-même.

Imaginons avoir écrit un programme qui, une fois compilé, se nomme echo ; supposons que ce programme soit lancé par la commande :

```
echo Pierre Paul
```

alors, main reçoit les arguments que montre la figure 1.

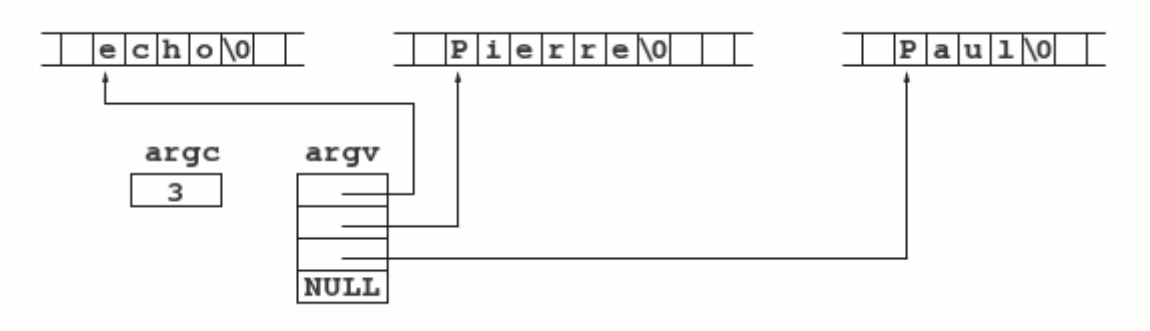


Fig. 1 - Arguments de main

Supposons que tout ce que l'on demande à echo soit de recopier la liste de ses arguments. Voici comment on pourrait écrire ce programme :

```
main(int argc, char *argv[]) {
int i;
for (i = 0; i < argc; i++)
printf("%s\n", argv[i]);
return 0;
}
```

La frappe de la commande « echo Pierre Paul » produit l'affichage de :

```
echo
Pierre
Paul
```

La principale application de ce mécanisme est la fourniture à un programme des paramètres dont il peut dépendre, comme des noms de fichiers, des tailles de tableaux, etc.

II) Manipulation de fichier

a) Ouverture de fichier

Lorsqu'on désire accéder à un fichier, il est nécessaire avant tout accès d'ouvrir le fichier à l'aide de la fonction *fopen*. Ci-dessous, un exemple de fonction permettant d'ouvrir un fichier :

```
FILE * Ouvrir_Fichier(char *nom_fichier, char *mode){
      FILE * fp = NULL; // fp pointeur de fichier

     if((fp=fopen(nomfich, mode)) == NULL){
            // Erreur ouverture de fichier
      printf("erreur d'ouverture du fichier %s\n",nomfich);
            abort();
      }
      else
            return(fp); // Sinon on retourne fp pointeur sur le fichier ouvert
}
```

- *nom-de-fichier* est de type pointeur vers `char`. La chaîne pointée est le nom du fichier auquel on veut accéder.

- *mode* est de type pointeur vers `char`. La chaîne pointée indique le mode d'ouverture, elle peut, par exemple, être l'une des chaînes suivantes :

  - `"r"` => ouverture d'un fichier texte en lecture.
  - `"w"` => ouverture d'un fichier texte en écriture.
  - `"a"` => ouverture d'un fichier texte en écriture à la fin.

b) Lecture d'un fichier

La fonction *scanf* permet de récupérer des données formatées sur le flux d'entrée standard (*stdin*). La fonction *fscanf* fait la même chose au détail près que nous pouvons spécifier le flux dans lequel la lecture se fera. La déclaration de cette fonction est la suivante :

```
#include <stdio.h>
int fscanf (FILE * stream, const char * format, ...);
```

La fonction renverra le nombre d'élément (et non de caractères) lus. Le deuxième argument et ceux qui le suivent servent à définir le format qui servira à récupérer la valeur des variables. Ci-dessous, un exemple d'utilisation :

```
#include <stdio.h>

int main() {

  int nombre = 0;
  FILE *f = fopen("nombre.txt", "r");

   if  (f == NULL) abort() ;

  fscanf(f, "%d", &nombre);
  printf ("Nombre lu : %d\n", nombre);

  fclose(f);
  return 0
}
```

c) Écriture dans un fichier

La fonction *fprintf* se base sur la fonction *printf*. La syntaxe de déclaration de *fprintf* est la suivante :

```
#include <stdio.h>
int fprintf (FILE *stream, const char *format, ...);
```

La fonction renvoie un nombre entier qui sera, comme dans toute fonction d'écriture, le nombre d'octet écrit. Le premier argument de la fonction est le pointeur vers le fichier dans lequel on souhaite écrire. Les arguments suivants seront la chaine de format et les données à écrire. Ci-dessous, un exemple d'application :

```
#include <stdio.h>
#include <string.h>

int main() {

int a = 0;
int b = 0;

FILE *f = fopen("log.txt", "a");

if (f == NULL) abort() ;
else fprintf (f, "Le fichier a ete ouvert\n");

printf ("Veuillez entrer a :");
scanf ("%d", &a);

fprintf (f, "L'utilisateur a entre la valeur %d pour a\n", a);

printf ("Veuillez entrer b :");
scanf ("%d", &b);

fprintf (f, "L'utilisateur a entre la valeur %d pour b\n", b);
fprintf (f, "La somme de %d et %d est donc %d\n", a, b, a+b);
fprintf (f, "Le programme se termine\n");

fclose(f);
return 0;
}
```

d) Autres fonctions

La bibliothèque *stdio.h* fournie d'autres fonctions pour lire et écrire dans un fichier selon les besoins. Vous trouverez facilement la liste de ces fonctions en effectuant une recherche sur internet.

TP Programmation en langage C
Debugger : aide mémoire

I) Généralités

Le **GNU Debugger** également appelé **gdb** est le débogueur standard du projet GNU. Gdb permet de déboguer un programme en cours d'exécution (en le déroulant instruction par instruction ou en examinant et modifiant ses données), mais il permet également un débogage *post-mortem* en analysant un fichier *core* qui représente le contenu d'un programme terminé anormalement.
L'interface de gdb est une simple ligne de commande, mais il existe des applications frontales qui lui offrent une interface graphique beaucoup plus conviviale. L'utilitaire **ddd** (que nous utiliserons en TP) permet de cliquer sur une ligne de code directement dans le listing pour y placer un point d'arrêt alors que gdb seul nécessite la saisie du numéro de ligne.

II) Utilisation

- Pour être pleinement exploitable par gdb, un programme doit être compilé avec l'option '**-g**' de **gcc**.
- Pour lancer le débugger en ligne de commande : `$ gdb mon_prog`
- Pour lancer le débugger en utilisant l'interface graphique ddd : `$ ddd mon_prog`
- Les commandes de base :

break`[fichier_source:]fonction`
Met un point d'arrêt au début de la fonction spécifiée. Lorsque l'exécution du programme rencontre un de ces points d'arrêt, le programme est stoppé et le programmeur peut entrer d'autres commandes au prompt du débogueur. L'exécution pourra ensuite être reprise grâce à la commande 'cont'. A la place d'un nom de fonction, on peut directement passer un numéro de ligne dans le code source à la commande 'break'.

Condition `numero test`
Permet de poser une condition sur un point d'arrêt précédemment défini par la commande '`break`'. Par exemple, '`condition 1 i==10`' n'activera le point d'arrêt numéro 1 que dans le cas où la variable i vaut 10.

run `[liste de paramètres]`
Démarre l'exécution du programme avec les éventuels paramètres spécifiés.

bt
Backtrace : demande à gdb d'afficher le contenu de la pile du programme. Synonyme de la commande '`where`'.

print `expression`
Affiche la valeur d'une expression. En particulier, cette commande peut être utilisée pour afficher la valeur d'une variable.

cont

Demande la reprise de l'exécution d'un programme stoppé.

next

Exécute la ligne suivante dans un programme suspendu. Si cette ligne contient des appels de fonctions, elles seront exécutées sans être elles-mêmes tracées.

nexti

Idem que la commande '`next`', mais le pas de progression n'est plus la ligne de programme, mais l'opcode assembleur (commande à réserver pour les cas les plus désespérés).

set `[option] [parametres]`

Cette option permet de changer la plus grande partie des paramètres de gdb ; parmi les principales options disponibles :

`environment` : permet de changer une variable d'environnement du programme exécuté.
`language` : permet de changer le langage source par défaut.
`variable` : permet de changer à la volée le contenu d'une variable.

step

Idem que la commande '`next`', mais les éventuelles fonctions appelées seront elles aussi tracées.

stepi

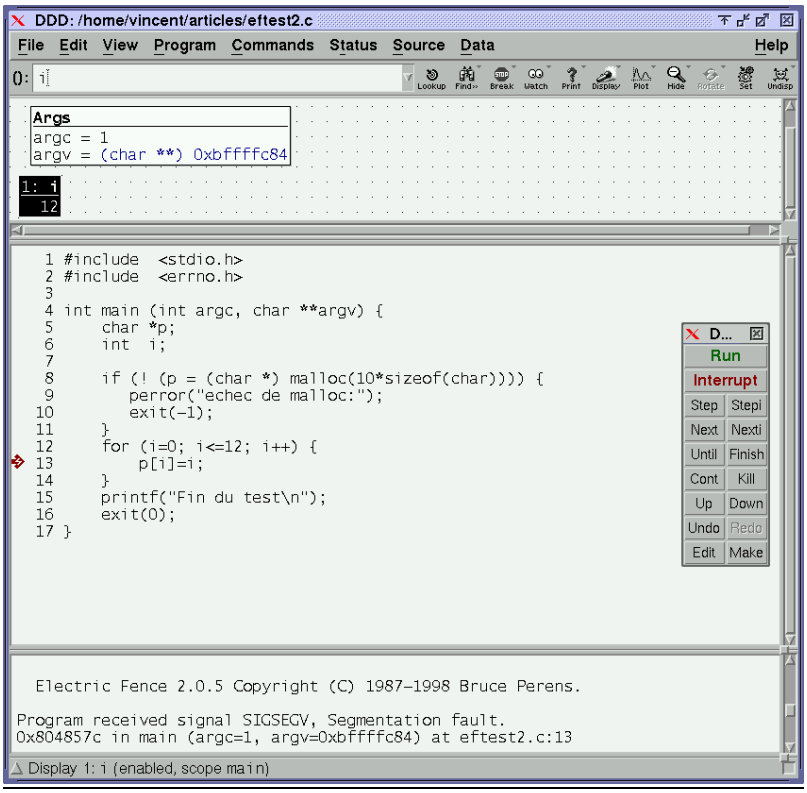
Idem que la commande '`step`', mais le pas de progression n'est plus la ligne de programme, mais l'opcode assembleur.

help `[nom]`

Donne des informations sur la commande donnée en paramètre, ou une page d'aide générique si aucun paramètre n'est mentionné.

quit

Quitte gdb.



makefile

```
# makefile

ARCH            = $(shell uname -s)-$(shell uname -m)

HOME            = .
BIN             = $(HOME)/bin/$(ARCH)
SRC             = $(HOME)/src
PROG            = $(BIN)/MonProg

#DEFS           = -DDEBUG
CFLAGS          = -g
CC              = gcc
LD              = gcc
LDFLAGS         = -lm
RM              = rm

OBJS            = $(BIN)/monprog.o $(BIN)/fonctions.o


$(PROG) : $(OBJS)
        @echo --- Edition des liens du programme monprog ---
        $(LD) $(LDFLAGS) $(OBJS) -o $(PROG)

$(BIN)/monprog.o : $(SRC)/monprog.c
        @echo --- Compile monprog.c---
        $(CC) $(CFLAGS) $(DEFS) -c $(SRC)/monprog.c -o $(BIN)/monprog.o

$(BIN)/fonctions.o : $(SRC)/fonctions.c
        @echo --- Compile fonctions.c ---
        $(CC) $(CFLAGS) -c $(SRC)/fonctions.c -o $(BIN)/fonctions.o


clean :
        @echo --- Nettoyage des fichiers temporaires ---
        @-$(RM) $(BIN)/*
```

TP Programmation en langage C n°1
Premiers pas

I) La compilation en langage C

- Le pré-processeur réalise plusieurs opérations de substitution sur le code C, notamment :
  - Suppression des commentaires (// et /* */)
  - Inclusion des fichiers .h dans le fichier .c (#include)
  - traitement des directives de compilation qui commencent par un caractère #. Ces directives sont: `define, elif, else, endif, error, if, ifdef, ifndef, include, line, pragma, undef.`
- Durant la phase de compilation, Le code du langage C est transformé en code Assembleur. Pour chaque fichier source, on obtient alors un fichier en langage assembleur.
- La phase d'assemblage permet de générer un fichier *objet* pour chaque fichier de code assembleur. Ces fichiers *objet* sont écrits en langage machine (format binaire).
- L'édition des liens est la dernière étape. Elle a pour but de réunir tous les éléments d'un programme. Les différents fichiers objets sont alors réunis, ainsi que les bibliothèques statiques, pour ne produire qu'un fichier exécutable.

II) La compilation avec GCC sous Linux

Synopsis : **gcc** [*option* | *fichier*]...

Les options de base de GCC:

`–help` : Affiche l'aide de GCC.
`–version` : Donne la version de GCC.
`-E` : Appelle le préprocesseur. N'effectue pas la compilation.
`-S` : Appelle le préprocesseur et effectue la compilation. N'effectue pas l'assemblage ni l'édition de lien. Seuls les fichiers assembleur (« .S ») sont générés.
`-c` : Appelle le préprocesseur, effectue la compilation et l'assemblage, mais ne fait pas l'édition de lien. Seuls les fichiers objets (« .o ») sont générés.
`-o nom` : Fixe le nom du fichier objet généré lors de la compilation d'un fichier source.
`-g` : Génère les informations symboliques de déboguage.
`-w` : Supprime tous les warnings.
`-W` : Active les warnings supplémentaires.
`-Wall` : Active tous les warnings possibles.

III) S’ouvrir au monde !

Ecrire un programme qui affiche à l’écran le texte : « Hello world »

IV) Priorité des opérateurs

Ecrire un programme qui réalise les calculs suivants et qui affiche chacune des valeurs de la variable entière X

X = -3 + 4 * 5 – 6
X = (-3 + 4) * 5 – 6
X = -3 + (4 * 5) – 6
X = -3 + 4 * (5 – 6)

V) La division

Ecrire un programme qui réalise les calculs suivants et qui affiche chacune des valeurs :

de la variable flottante puis entière X = 10 / 3
de la variable flottante X = 10.0 / 3
de la variable flottante X = 10.0 / 3.0
de la variable flottante X = (float) 10 / 3
de la variable flottante X = ((float) 10) / 3
de la variable flottante X = ((float) 10) /3 (affichage avec 1 puis 5 chiffres avec la virgule)

Expliquer les différences

V) Le modulo

Ecrire un programme qui réalise les calculs suivants et qui affiche chacune des valeurs :

de la variable entière X = 5 % 2
de la variable flottante X = 5.0 % 2

VI) Opérateurs logiques et Booléens

Ecrire un programme qui réalise les calculs suivants et qui affiche chacune des valeurs :

5 & 6
5 && 6
5 | 6
5 || 6
5 ^ 6
5 ^^ 6
~5
! 5

Expliquer les résultats obtenus.

VII) Autour du cercle

Ecrire un programme qui récupère la valeur flottante du rayon d'un cercle et qui en affiche le périmètre et la surface. La constante PI aura la valeur 3.14159

VIII) Calculatrice

Ecrire un programme qui récupère le type d'une opération (/, *, -, +) puis les valeurs de deux opérandes (flottantes) et qui affiche le résultat. Signaler une erreur de saisie lorsque le type de l'opération n'est pas défini ou lorsque l'opération ne peut être réalisée. Le programme devra boucler jusqu'à qu'un caractère 's' soit saisi.

1 en utilisant des instructions *if*
2 en utilisant une instruction *case*

IX) Somme des n premiers entiers

Ecrire un programme qui calcule et affiche la somme des n premiers entiers avec

Une boucle *for* à indice incrémenté
Une boucle *for* à indice décrémenté
Une boucle *while* à indice incrémenté
Une boucle *while* à indice décrémenté

X) PGCD

Ecrire un programme qui calcule le plus grand commun diviseur de deux nombres entiers a et b saisis par l'utilisateur en appliquant l'algorithme suivant : on remplace le plus grand par le résultat de la soustraction du plus grand et du plus petit jusqu'à que les deux nombres soient égaux.

XI) Calcul de X^N

Écrire un programme qui lit deux nombres entiers X et N et calcule X^N en testant préalablement si X est strictement positif et N positif.

XII) Voyelle

Ecrire un programme qui récupère un texte qui se termine par le caractère '#' et qui affiche le nombre de voyelle qu'il contient

TP Programmation en langage C n°2
Structures de contrôle

II) Etoile

Ecrire un programme qui permet :
- de saisir un entier N avec 1≤N≤20 (N = nombre de lignes constituées d'étoiles)
- de proposer un menu qui affiche à l'écran la disposition 1, 2, 3, 4 ou 5 :

Pour un entier N=5

1)
```
*****
*****
*****
*****
*****
```

2)
```
*
**
***
****
*****
****
***
**
*
```

3)
```
    *
   **
  ***
 ****
*****
 ****
  ***
   **
    *
```

4)
```
  *
 ***
*****
*******
*********
```

5)

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

Contraintes : utiliser au moins une fois chacune de ces trois structures : SWITCH, FOR, WHILE.

II) Algorithme de tri

On dispose de N=30 valeurs numériques que l'on désire classer par ordre croissant (on aurait pu choisir décroissant). Chacune de ces valeurs sera choisie aléatoirement dans l'intervalle [0 ;50] et placée dans un tableau TAB de taille N.

Faites 3 programmes qui effectuent les 3 méthodes de tri suivantes et affichent à l'écran le tableau TAB en ligne puis en dessous le tableau TAB trié :

1) Tri bulles

On compare les couples de valeurs TAB[i] et TAB[i+1] pour i variant de 0 à N-2 :

Si TAB[i]>TAB[i+1], on les permute.

L'algorithme s'arrête quand il n'y a plus de permutations à effectuer : le tableau TAB est alors trié.

2) Tri par recherche des minima successifs

Dans un premier passage, on recherche le minimum du tableau TAB que l'on permute avec TAB[0].

Puis sur les N-1 éléments restants, on cherche le deuxième minimum que l'on permute avec TAB[1].

Etc…

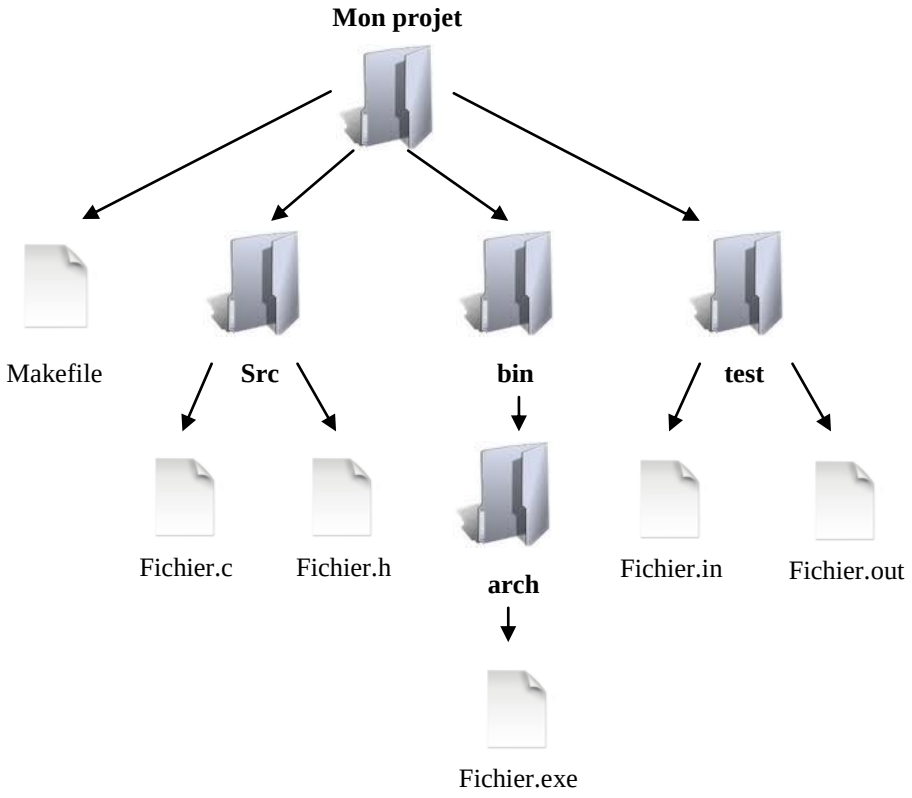
3) Tri par insertion

Insérer une valeur V à la place j d'un tableau déjà trié consiste à décaler vers la droite tous les éléments du tableau à partir de la place i puis à faire T[j]=V.

Pour trier le tableau TAB, il suffit alors d'insérer TAB[i] dans les i-1 premiers éléments déjà triés et placé dans un autre tableau, ceci pour i variant de 1 à N-1.

TP Programmation en langage C n°3
Les fonctions

I) Structurer son environnement



Remarque : Un exemple de MAKEFILE est disponible en début de document. Vous adapterez cet exemple aux différents exercices de ce TP.

II) Les bases

Ecrire un programme affiche «Hello world » à l'aide d'une fonction *affiche_hello()* en

1. Définissant l'entête et le corps de la fonction avant le *main*
2. Définissant l'entête de la fonction avant le *main* et le corps de la fonction après le main
3. Définissant l'entête et le corps de la fonction dans un fichier *fonction.h*
4. Définissant l'entête de la fonction dans un fichier *fonction.h* et le corps de la fonction dans un fichier *fonction.c*

III) Horloge

Ecrire une fonction qui convertit le temps exprimé sous forme heures, minutes et secondes en secondes.

IV) Les types de retour des fonctions

Ecrire un programme qui à l'aide d'une fonction *compare(…)* compare deux entiers fournis par l'utilisateur et qui retourne :

1. La valeur entière 1 si les entiers sont égaux, 0 sinon
2. Le caractère V si les entiers sont égaux, F sinon.

Même question avec la comparaison de deux flottants.

Le programme principal devra afficher la valeur retournée par la fonction.

V) Conversions degrés

Les conversions des degrés Celsius à Kelvin et des degrés Celsius à Fahrenheit sont données par les équations suivantes :

$$K = C + 273$$
$$F = (9/5)\ C + 32$$

1. Ecrivez un module de conversion contenant toutes les fonctions de conversion entre ces trois échelles(*CelsiusAKelvin, CelsiusAFahrenheit*, …) ; ce module sera découpé en deux fichiers ***degre.h*** et ***degre.c***, comme il se doit.

2. Utilisez ensuite ce module dans un programme ***testdegre1.c*** pour tester rapidement les fonctions du module.

3. Améliorez la première version en écrivant un programme ***testdegre2.c*** (avec menu). Cette version comportera les fonctions suivantes :

   - ***int lireChoix()***
   Affiche le menu précisant les conversions possibles et retourne le choix de la conversion voulue.

   - ***void executerChoix(int choix)***
   Saisit la température à convertir puis exécute la conversion selon le choix passé en paramètre.

   - ***void main()***
   Fonction principale qui fait appel aux deux fonctions précédentes. On boucle tant que le choix de l'utilisateur est différent de 0.

TP Programmation en langage C n°4
Structures

I) Déclarations

La déclaration d'un tel type doit être de la forme suivante :

```
TYPEDEF STRUCT {
        déclaration_champs1 ;
        déclaration_champs2 ;
        .
        déclaration_champsN ;
        } nom_type ;
```

Où nom_type est le nom que l'on donne au type structuré, déclaration_champs1, déclaration_champs2, déclaration_champsN représentent les déclarations des différents champs qui vont composer la variable structurée. Ces déclarations doivent donc être de la forme : type_du_champs nom_du_champs.

```
Void main (void){

Nom_type variable ;
// ou bien :
struct nom_de_type
        {
        déclaration des champs
        } variable ;
        …
// Toute référence à un champ de variable structurée doit avoir la forme :
nom_variable.nom_champs
}
```

II) Carnet d'adresse

Considérons l'exemple suivant :
Une personne possède un certain nombre de caractéristiques : un nom, un prénom, une adresse, un âge, etc …
On regroupe ces caractéristiques dans une variable structurée « PERSONNE », on pourra ainsi accéder à la variable globalement ou par l'intermédiaire d'une de ses composantes.

Structure PERSONNE :

⇒ Nom : 20 caractères

⇒ Prénom : 20 caractères

⇒ Date de naissance (structure DATE):
- ◊ Jour : entier
- ◊ Mois : entier
- ◊ Année : entier

Ecrire un programme constitué des fonctions suivantes :
- Une fonction qui permet de définir une personne en entrant ses caractéristiques au clavier.
- Une fonction permettant l'affichage des données déjà entrées.
- Une fonction permettant de classer les noms par date de naissance.
- Une fonction permettant de retrouver, modifier et afficher les données relatives à une personne à partir de son nom.
- Une fonction permettant de supprimer un enregistrement.

Les données seront rangées dans un tableau de dimension 20 (nombre maximum de personnes pouvant être définies) et un menu permettra le choix entre les différentes fonctions et la sortie du programme.
Le tableau de personnes sera déclaré comme une variable globale.

TP Programmation en langage C n°5
Pointeurs et introduction aux listes chainées

I) Les variables dévoilent pour nous leurs adresses – L'opérateur &

Ce premier exercice a pour objectif de démontrer la signification de l'opérateur & appliqué à une variable ou à un élément de tableau. Nous vous proposons tout d'abord de saisir, compiler et exécuter le petit programme suivant :

```
/*
 *   TP 5 : Pointeurs
 *
 *   lecture de l'adresse d'une variable
 */

#include <stdio.h>

void main()
{
  int i,j;

  i = 5;
  j = 12 ;

  printf("La variable i est située à l'adresse %u et vaut %d\n",&i,i);
  printf("La variable j est située à l'adresse %u et vaut %d\n",&j,j);
}
```

Nous indiquons à la fonction « printf » que l'adresse « &i » de la variable « i » doit être affichée sous la forme d'un entier décimal **positif** à l'aide du formateur « %u » (**unsigned int**). Pour afficher les adresses en notation hexadécimale, il suffit de remplacer « %u » par « %x ».
La mémoire de l'ordinateur peut être considérée comme un tableau. Déclarer une variable de type « int » revient à réserver une case dans ce tableau pour le stockage d'un nombre entier. L'adresse de cette variable correspondra alors à l'indice dans le tableau mémoire de la variable.

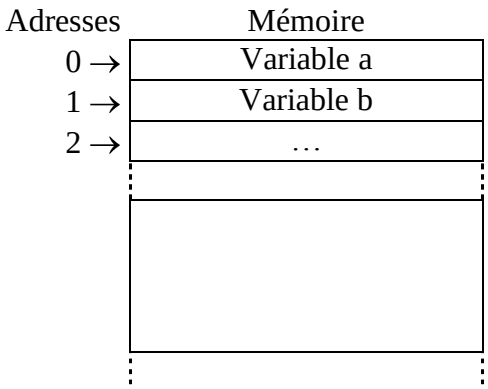


Figure 1. Représentation en mémoire

Le programme suivant illustre le stockage en mémoire d'un tableau.

```
/*
 *   TP 5 : Pointeurs
 *
 *   : lecture de l'adresse des éléments d'un tableau
 */

#include <stdio.h>
#define N 5
void main()
{
  int i;
  int t[N] = {10, 20, 30, 40, 50};

  for(i=0;i<N;i++)
    printf("t[%d] = %d ; &t[%d] = %u\n",i,t[i],i,&t[i]);
}
```

Questions

1. Qu'observe-t-on lors de l'exécution de ce programme ?
2. Illustrez sous forme d'un schéma (plan mémoire) similaire à celui de la Figure 1 la représentation en mémoire du tableau « t ».
3. Compilez et exécutez plusieurs fois ce programme en remplaçant dans la déclaration du tableau « t » le type « int » par « short int », « float », « double ». Que peut-on en déduire sur le format de stockage des types de données mentionnés ?
4. Écrire le programme suivant et expliquer à l'aide d'un schéma son fonctionnement et sa représentation mémoire.

```
void main(void)
{

int *px;
px = (int *) malloc (sizeof(int));

*px = 1;

printf("Adresse de px : %X\n", &px);
printf("Adresse pointee par le pointeur px = %X\n", px);
printf("Valeur memorisee a l'adresse pointee par le pointeur px = %d\n\n",
  *px);
```

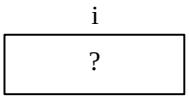
II) Accéder au contenu des variables de manière indirecte: L'opérateur *

A l'inverse des **variables**, dont le rôle est de permettre la manipulation de **données**, les pointeurs fournissent un moyen de manipuler les **adresses**. L'opérateur « * », appelé « opérateur d'indirection » ou « opérateur de déréférence », permet d'accéder à une donnée stockée en mémoire en connaissant son type et son adresse.

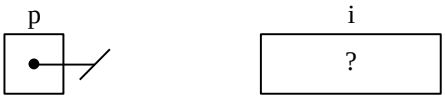
```c
/*
 *   TP 5 : Pointeurs
 *
 *   accès au contenu d'une variable
 *   par l'intermédiaire d'un pointeur
 */

#include <stdio.h>

void main()
{
  int i;        /* (1) */
  int *p;       /* (2) */

  i = 5;
  p = &i;       /* (3) */

  printf("Le pointeur p designe la donnée située à l'adresse %u\n",p);
  printf("La donnée pointée par p vaut %d\n",*p);

  *p = 12 ;     /* (4) */

  printf("La variable i contient la valeur %d\n",i);
}
```

L'exécution du programme ci-dessus peut se représenter graphiquement de la manière suivante. Les numéros des étapes successives renvoient aux numéros placés en commentaire dans le source C :

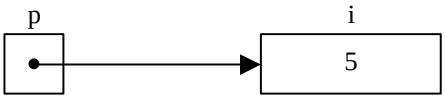
1. Création d'une variable entière nommée « i ». Cette variable occupe 4 octets en mémoire et se trouve à une certaine adresse « &i » arbitrairement choisie au moment de l'exécution :



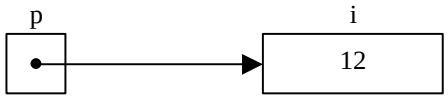
2. Création d'un pointeur d'entier nommé « p ». Lors de sa déclaration, ce pointeur prend la valeur prédéfinie « NULL ». Autrement dit, ce pointeur ne pointe sur rien du tout.



3. Affectation de la valeur 5 à la variable « i ». L'adresse de « i » est affectée au pointeur « p ». On dit que « p » « pointe sur i ». Graphiquement, nous représentons cela de la manière suivante :



4. Modification de la valeur de la donnée située à l'adresse contenue dans « p ». Puisque cette adresse est celle de « i », c'est bien la valeur de « i » que se trouve modifiée :



Question

Représentez schématiquement les étapes successives de l'exécution du programme suivant :

```
#include <stdio.h>

void main()
{
  int i,j,k,tmp;
  int *pi,*pj,*pk,*ptmp;

  i=31 ; j=52 ; k=17;
  pi=&i ; pj=&j ; pk=&k;

  printf("i=%d j=%d k=%d\n",i,j,k);
  printf("*pi=%d *pj=%d *pk=%d\n",*pi,*pj,*pk);

  tmp=*pi;
  *pi=*pj;
  *pj=*pk;
  *pk=tmp;

  printf("i=%d j=%d k=%d\n",i,j,k);
  printf("*pi=%d *pj=%d *pk=%d\n",*pi,*pj,*pk);

  ptmp=pi;
  pi=pj;
  pj=pk;
  pk=ptmp;

  printf("i=%d j=%d k=%d\n",i,j,k);
  printf("*pi=%d *pj=%d *pk=%d\n",*pi,*pj,*pk);
}
```

III) Écrire une procédure qui modifie la valeur de ses arguments

Nous nous proposons d'écrire une procédure qui échange la valeur de deux entiers fournis en argument. Une première implémentation, naïve et incorrecte est donnée ci-après :

```
/*
 *   TP 5 : Pointeurs
 *
 *   insuffisance des appels de fonction
 *   avec passage par valeur
 */

#include <stdio.h>

void echanger(int a, int b)
{
     int tmp;
```

```
	tmp = a;
	a = b;
	b = tmp;
}

void main()
{
	int x,y;

	x = 12;
	y = 34;

	printf("Avant échange : x = %d ; y = %d\n",x,y);

	echanger(x,y);

	printf("Après échange : x = %d ; y = %d\n",x,y);
}
```

Questions

1. Expliquer, à l'aide de schémas et en détaillant les étapes de l'exécution, pourquoi le programme ci-dessus ne fonctionne pas.
2. Corriger ce programme en remplaçant les paramètres entiers « a » et « b » de la procédure « échanger » par des pointeurs d'entiers « pa » et « pb ».
3. Expliquer, à l'aide de schémas et en détaillant les étapes de l'exécution, pourquoi le programme corrigé fonctionne.

III) Introduire des doubles pointeurs

Expliquer le fonctionnement de la procédure suivante. Écrire une fonction « main » similaire à celle proposée dans III), et qui illustre l'utilisation de cette procédure.

```
void echangerPointeurs(int **ppa, int **ppb)
{
	int *tmp;

	tmp = *ppa;
	*ppa = *ppb;
	*ppb = tmp;
}
```

IV) *met en application la notion de pointeur* – Listes chaînées

Les listes chaînées appartiennent à la famille des structures récursives. Le caractère récursif de ce type de structure peut être mis en évidence à l'aide de la définition suivante :
Une **liste** d'éléments (valeurs entières, chaînes de caractères, par exemple) peut être :

- soit vide
- soit constituée d'un élément initial suivi d'une **liste** (éventuellement vide) des éléments suivants.

En langage C, on accède à une liste par l'intermédiaire d'un pointeur

- ce pointeur vaut « NULL » si la liste est vide (Figure 2)
- si la liste n'est pas vide, ce pointeur donne accès à un objet structuré de type « maillon » (Figure 3), contenant d'une part le premier élément de la liste considérée, et d'autre part un pointeur sur la suite de la liste (qui est elle-même une liste).

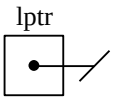


Figure 2. Une liste vide

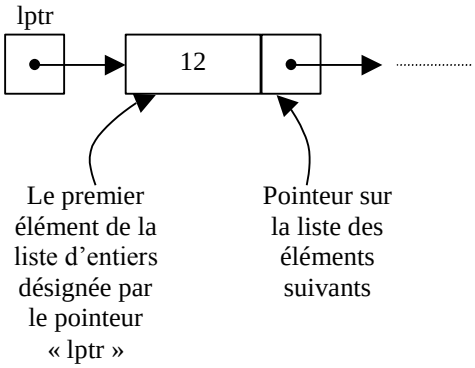


Figure 3. Maillon élémentaire constituant une liste chaînée non vide

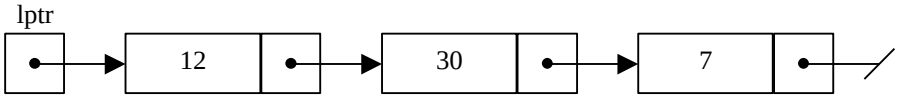


Figure 4. Exemple : liste chaînée de trois entiers

A la différence des tableaux, qui, en langage C, ont une taille figée et doivent contenir au moins un élément, les structures de type liste chaînée ont une longueur variable (on peut aisément ajouter, insérer, supprimer des éléments) et peuvent être éventuellement vides.
Nous allons à présent nous intéresser à l'implémentation en langage C de fonctions de manipulation d'une liste de noms.

1. Écrire le programme suivant et expliquer à l'aide d'un schéma son fonctionnement et sa représentation mémoire.

```
#define N 3

typedef struct maillon {
    int x;
    struct maillon * suiv;
    } maillon;


void main(void)
{

maillon lc;
lc.x = 1;

printf("Valeur du champs x = %d\n\n", lc.x);
lc.suiv = (maillon *) malloc(sizeof(maillon));
lc.suiv->x = 2;
printf("Valeur du champs x du deuxieme maillon = %d\n\n", lc.suiv->x);

}
```

2. Ecrire le programme suivant et expliquer à l'aide d'un schéma son fonctionnement et sa représentation mémoire.

```
…
void main(void)
{

maillon *lc;
maillon *tete;
int cpt;

/*init des maillons*/
lc = (maillon *) malloc(sizeof(maillon));
tete = lc;

//Creation des maillons en fin de liste
for(cpt=1;cpt<N;cpt++) //pour tous les maillons à créer
{
    lc->suiv = (maillon *) malloc(sizeof(maillon));
    lc = lc->suiv;
}
lc->suiv = NULL;

cpt = 0;
lc = tete;

//remplissage des valeurs de chacun des maillons
while (lc != NULL) //tant que le maillon courant n'est pas le suivant du
dernier maillon de la liste
    {
          lc->x = cpt; //affectation
          cpt++; //increment du compteur
          lc = lc->suiv;//passe au suivant
    }

lc = tete;

while (lc != NULL) //tant que le maillon courant n'est pas le suivant du
dernier maillon de la liste
    {
          printf("Valeur du champs courant = %d \n",lc->x);
          printf("Adresse maillon courant= %X et du suivant %X\n",lc, lc-
>suiv);
          lc = lc->suiv;//passe au suivant
    }

}
```

3. Ecrire un programme qui construit une liste chainée de N maillons. La valeur de N sera définie dans une variable globale. Ecrire une fonction permettant de supprimer un maillon de la liste chainée sans briser le chainage. Le numéro du maillon à supprimer sera demandé à l'utilisateur.

Remarque :

Dans le programme qui précède, nous avons abondamment utilisé l'opérateur « -> » pour accéder aux champs d'un maillon référencé par un pointeur. Rappels syntaxiques :

- ordinairement, la sélection d'un champ d'un objet de type structuré se fait au moyen de l'opérateur « . ». Par exemple « m.element » si « m » est une variable de type « Maillon ».
- si la donnée structurée est référencée par un pointeur, on peut utiliser l'opérateur d'indirection : si « lc» est un pointeur de « Maillon », on écrira « (*lc).element ». Les parenthèses sont obligatoires, si vous les oubliez, le compilateur interprétera « *lc.element » comme « *(lc.element) », ce qui n'a pas du tout la même signification.
- le langage C propose une syntaxe condensée, strictement équivalente à « (*lc).element », sous la forme « lc->element »

TP Programmation en langage C n°6
Listes chainées

I) Contrôle d'accès

Les systèmes de contrôle d'accès peuvent être constitués d'un lecteur de badges relié avec un micro-ordinateur. Sur le micro-ordinateur un programme va permettre de vérifier le numéro d'identification des badges, et l'authentification.

Écrire un programme pour saisir une liste de personnes dédiée au control d'accès. Chaque personne est définie par son nom, son prénom, son numéro de badge (4 chiffres), son code secret et enfin la date et l'heure de son dernier passage au contrôle d'accès.

Ecrire un programme pour réaliser les **fonctions** suivantes au travers d'un menu.

1) Afficher la liste des personnes enregistrées.
2) Ajouter une personne à la liste.
3) Supprimer une personne suivant le numéro de badge.
4) Modifier le code secret d'une personne.
5) Simuler le contrôle d'accès via le clavier :
   - saisir le numéro de badge.
   - demander le code secret correspondant au badge.
   - suivant le code secret :
     *Afficher le nom de la personne et inscrire la date et l'heure de la visite.
     *Afficher accès refusé

Remarque : On créera une structure de donnée permettant de stocker les informations définissant une personne. La liste de personnes sera construite à l'aide d'une liste doublement chainée. De plus, aucune variable globale ne pourra être utilisée.

6) Programmer une fonction permettant de sauvegarder la liste de personnes dans un fichier.
7) Écrire une fonction qui permet de lire un fichier contenant une liste de personnes et de stocker ces informations dans une liste doublement chainée pouvant être manipulée par les fonctions précédemment écrites.

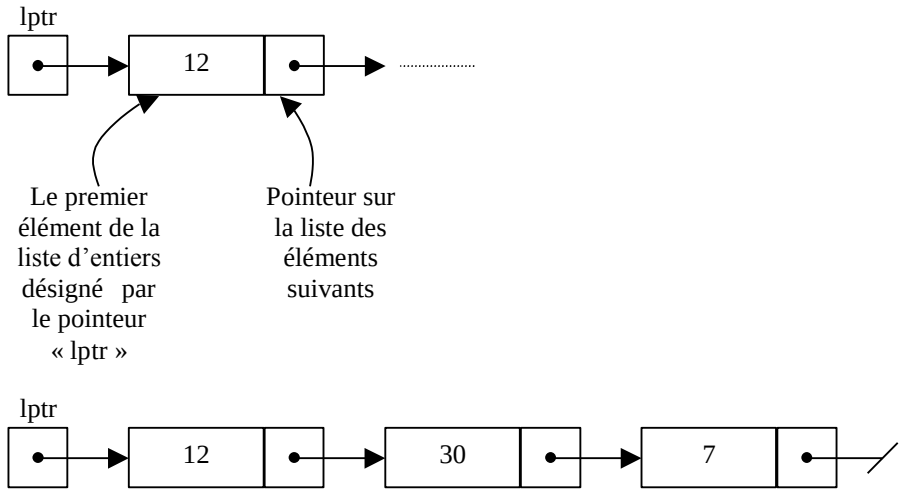
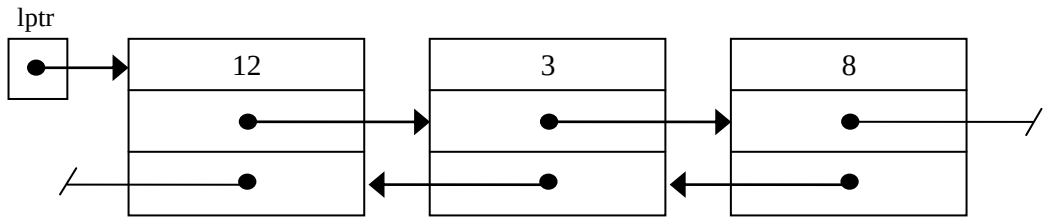
Remarque : Pour cela on utilisera les fonctions *fopen*, *fprintf* et *fscanf*.

Annexe 1 : Date et Heure, utilisation de time.h

```
#include <stdio.h>
#include <time.h>

int main(void)
{
  time_t timer = time(NULL);
  printf("current time is %s", ctime(&timer));
  system("PAUSE");
  return 0;
}
```

Annexe 2 : Liste doublement chainée

Rappel : liste simplement chainée (cf TP n°5)



Une liste doublement chainée possède un pointeur sur la liste des éléments suivants mais aussi un pointeur sur la liste des éléments précédents. La figure ci-dessous représente une liste doublement chainée de 3 éléments. Le pointeur « précédent » du premier élément et le pointeur « suivant » du dernier élément sont initialisés à **NULL**.